Dictionaries and Sets

Software Development COS7009-B

Contents

Dictionary
Dictionary Applications
Set
Set Applications
Build Dictionary and Set faster

What is a dictionary?

- In data structure terms, a dictionary is better termed an associative array, associative list or a map.
- You can think if it as a list of pairs, where the first element of the pair, the key, is used to retrieve the second element, the value.
- Thus we map a key to a value

Python Dictionary

- Use the { } marker to create a dictionary
- Use the : marker to indicate key: value pairs

```
contacts= {'bill': '353-1234',
 'rich': '269-1234', 'jane': '352-1234'}
print contacts
{'jane': '352-1234',
 'bill': '353-1234',
 'rich': '369-1234'}
```

Keys and Values

- Key must be immutable
  - strings, integers, tuples are fine
  - lists are NOT
- Value can be anything

Collections but Not a Sequence

- dictionaries are collections but they are not sequences such as lists, strings or tuples
  - there is no order to the elements of a dictionary
  - in fact, the order (for example, when printed) might change as elements are added or deleted.
- So how to access dictionary elements?

Access dictionary elements

Access requires `[ ]`, but the *key* is the index!

```
my_dict={}
```

– an empty dictionary

```
my_dict['bill']=25
```

added the pair 'bill':25

```
print(my_dict['bill'])
```

Dictionaries are mutable

- Like lists, dictionaries are a mutable data structure
  - you can change the object via various operations, such as index assignment

```
my_dict = {'bill':3, 'rich':10}

print(my_dict['bill'])     # prints 2

my_dict['bill'] = 100

print(my_dict['bill'])     # prints 100
```

Fewer Methods

- Only 9 methods in total. Here are some
  - `key in my_dict`
    -does the key exist in the dictionary
  - `my_dict.clear()` - empty the dictionary
  - `my_dict.update(yourDict)` - for each key in `yourDict`, updates `my_dict` with that key/value pair
  - `my_dict.copy` - shallow copy
  - `my_dict.pop(key)` - remove key, return value

Dictionary content methods

- `my_dict.items()` – all the key/value pairs
- `my_dict.keys()` – all the keys
- `my_dict.values()` – all the values
- These return what is called a **dictionary view.**
- the order of the views corresponds are dynamically updated with changes are iterable

`my_dict.get(key, default)`
–

The behavior of `get` is to provide normal dictionary operations when the key exists (return the associated value), but if the key does not exists, it returns the default as the value for that key (no error).

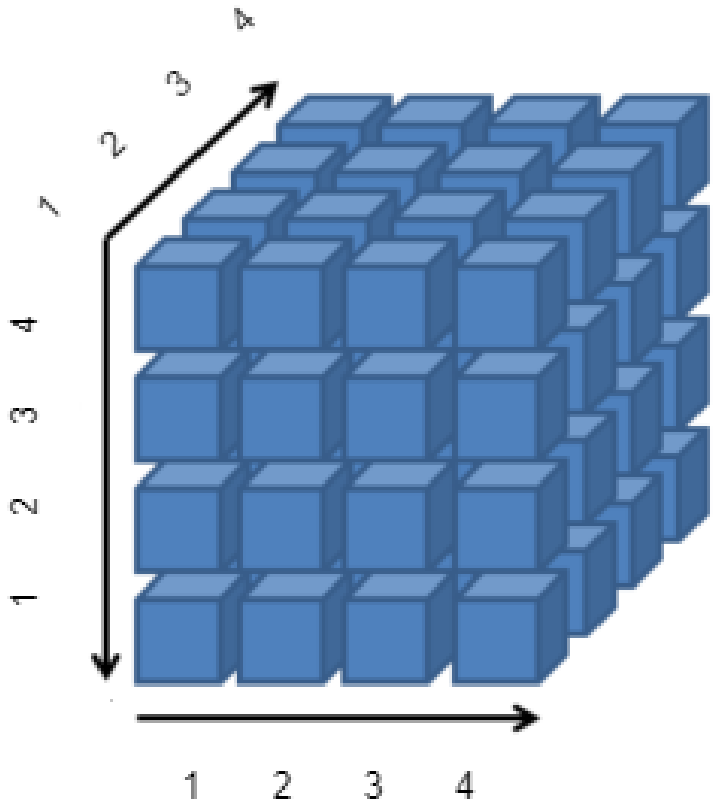
Contents

Dictionary
Dictionary Applications
Set
Set Applications
Build Dictionary and Set faster

Application 1

- Imagine you are trying to count the frequency of word occurrence in a list of words (perhaps gathered from a text file).We can represent those data using a dictionary, with the words as the keys and the values as the frequency of occurrence. There are two operations you need to perform in the process of updating a dictionary with a word from the list.
  - You can add a word to the dictionary if it doesn't exist (with a frequency of one in that case)
  - or you can add one to the frequency of an existing word already in the dictionary.

Application 2



- Consider a problem of implementing a sparse data structure. For example, the 3D data on the left only have a few position have values in them, e.g.
- At (2,3,4), the value is 55
- At (1,2,3), the value is 66

Contents

Dictionary
Dictionary Applications
Set
Set Applications
Build Dictionary and Set faster

Sets, as in Mathematical Sets

- in mathematics, a set is a collection of objects, potentially of many different types
- in a set, no two elements are identical. That is, a set consists of elements each of which is unique compared to the other elements
- there is no order to the elements of a set
- a set with no elements is the empty set

Creating a Set

Set can be created in one of two ways:

- constructor: `set(iterable)` where the argument is iterable

```
my_set = set('abc')
my_set → {'a', 'b', 'c'}
```

- shortcut: `{}`, braces where the elements have no colons (to distinguish them from dicts)

```
my_set = {'a', 'b','c'}
```

Diverse elements

- A set can consist of a mixture of different types of elements

```
my_set = {'a',1,3.14159,True}
```

  - as long as the single argument can be iterated through, you can make a set of it

No Duplicates

- duplicates are automatically removed

```
my_set = set("aabbccdd")

print(my_set)
```

→ `{'a', 'c', 'b', 'd'}`

Common Operators

Most data structures respond to these:

- `len(my_set)`
  – the number of elements in a set
- `element in my_set`

  boolean indicating whether element is in the set

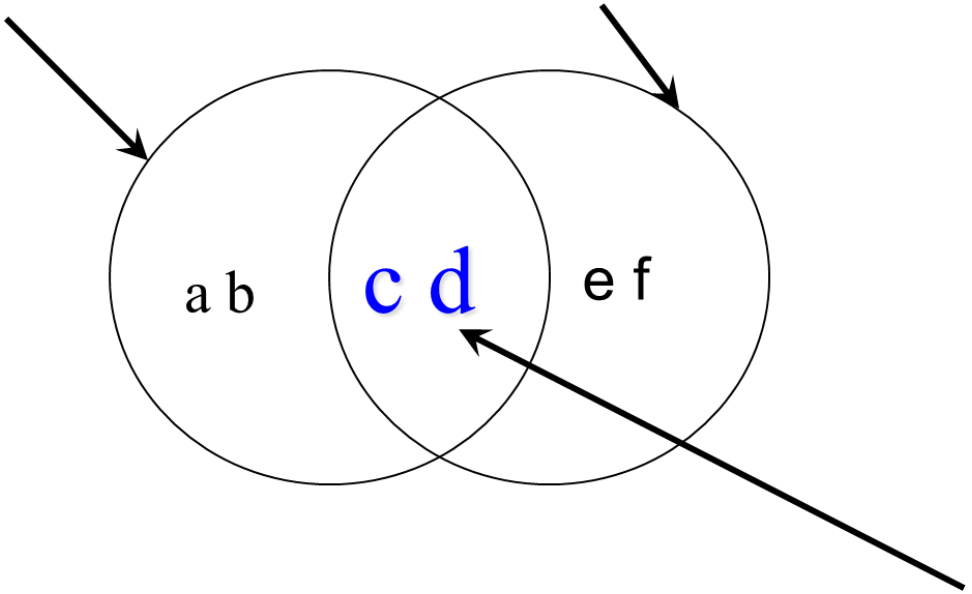
- `for element in my_set:`
  – iterate through the elements in

Contents

Dictionary
Dictionary Applications
Set
Set Applications
Build Dictionary and Set faster

Method: Intersection, Op colon ampersand

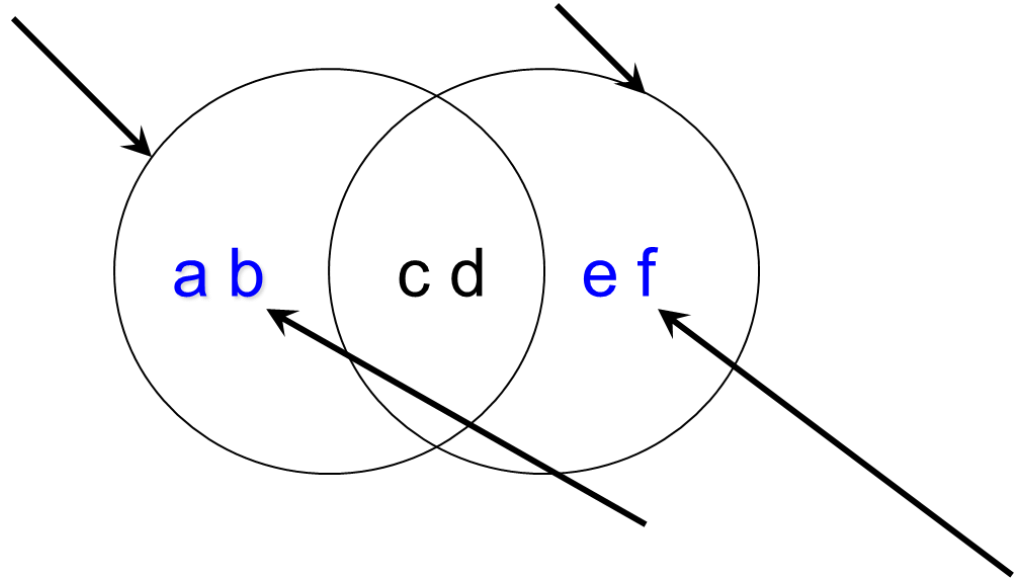
a_set=set("abcd") b_set=set("cdef")




```
a_set & b_set → {'c', 'd'}
b_set.intersection(a_set) → {'c', 'd'}
```

Method: Difference Op:-

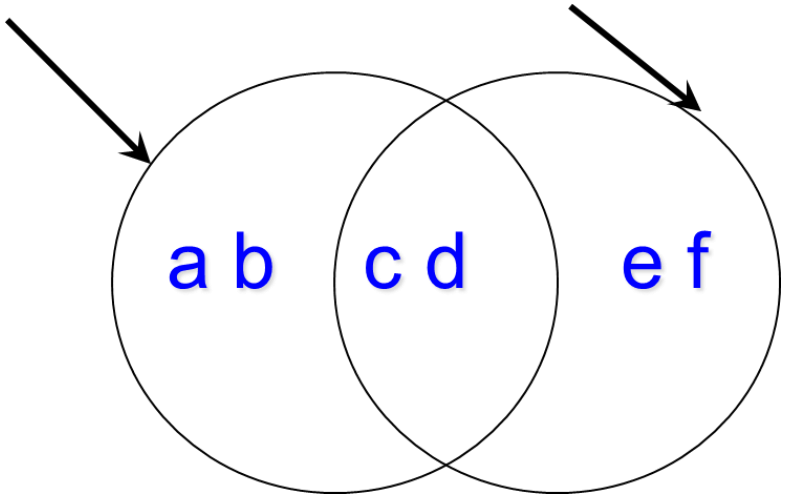
a_set=set("abcd") b_set=set("cdef")




```
a_set - b_set → {'a', 'b'}
b_set.difference(a_set) → {'e', 'f'}
```

Method: Union, OP colon pipe

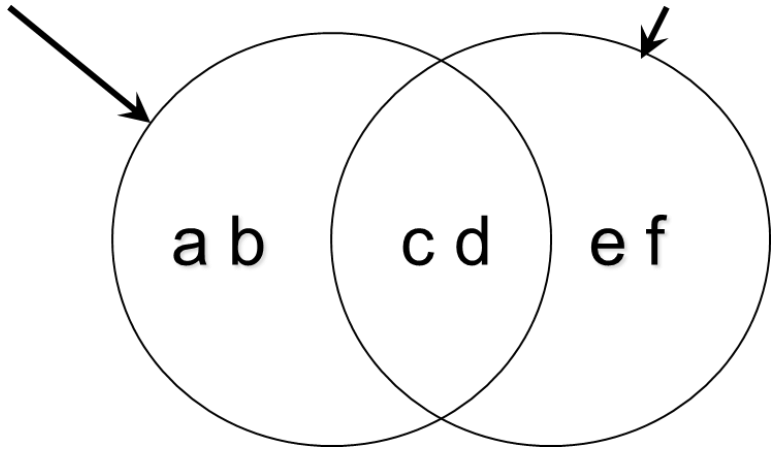
a_set=set("abcd") b_set=set("cdef")




```
a_set | b_set → {'a', 'b', 'c', 'd', 'e', 'f'}
b_set.union(a_set) → {'a', 'b', 'c', 'd', 'e', 'f'}
```

Method: symmetric_difference, OP colon Caret.

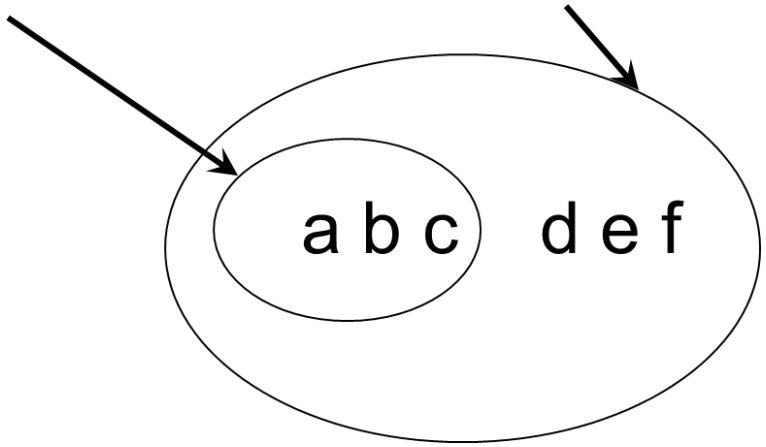
a_set=set("abcd"); b_set=set("cdef")




```
a_set ^ b_set → {'a', 'b', 'e', 'f'}
b_set.symmetric_difference(a_set) → {'a', 'b', 'e', 'f'}
```

Method: Issubset, OP colon less than equals
Method: Issuperset, OP colon greater than equals

small_set=set("abc"); big_set=set("abcdef")




```
small_set <= big_set → True
bit_set >= small_set → True
```

Other Set Ops

- `my_set.add("g")`
  - adds to the set, no effect if item is in set already
- `my_set.clear()`
  - emptys the set
- `my_set.remove("g")` versus

  `my_set.discard("g")`
  - `remove` throws an error if "`g`" isn't there. `discard` doesn't care. Both remove "`g`" from the set
- `my_set.copy()`

  returns a shallow copy of `my_set`

Contents

Dictionary
Dictionary Applications
Set
Set Applications
Build Dictionary and Set faster

Building dictionaries faster

- `zip` creates pairs from two parallel lists
  - `zip("abc",[1,2,3])` yields

```
[('a',1),('b',2),('c',3)]
```

- That's good for building dictionaries. We call the `dict` function which takes a list of pairs to make a dictionary
  - `dict(zip("abc",[1,2,3]))` yields
  - `{'a': 1, 'c': 3, 'b': 2}`

Dict and Set Comprehensions

Like list comprehensions, you can write shortcuts that generate either a dictionary or a set, with the same control you had with list comprehensions

- both are enclosed with {} (remember, list comprehensions were in [])
- difference is if the collected item is a : separated pair or not

Dict Comprehension

```
>>> a_dict = {k:v for k,v in enumerate('abcdefg')}
>>> a_dict
{0: 'a', 1: 'b', 2: 'c', 3: 'd', 4: 'e', 5: 'f', 6: 'g'}
>>> b_dict = {v:k for k,v in a_dict.items()} # reverse key—value pairs
>>> b_dict
{'a': 0, 'c': 2, 'b': 1, 'e': 4, 'd': 3, 'g': 6, 'f': 5}
>>> sorted(b_dict)      # only sorts keys
['a', 'b', 'c', 'd', 'e', 'f', 'g']
>>> b_list = [(v,k) for v,k in b_dict.items()]    # create list
>>> sorted(b_list)                                # then sort
[('a', 0), ('b', 1), ('c', 2), ('d', 3), ('e', 4), ('f', 5), ('g', 6)]
```

Set Comprehension

```
>>> a_set = {ch for ch in 'to be or not to be'}
>>> a_set
{' ', 'b', 'e', 'o', 'n', 'r', 't'}  # set of unique characters
>>> sorted(a_set)
[' ', 'b', 'e', 'n', 'o', 'r', 't']
```

Reference

- Punch & Enbody Chap 9